Page No.
Date

Assignment - 8

.1 freq greater than k.

```
int arr[] = { 1,1,2,2,2,3 }
int target = 2;
for (int i=0; i<arr.length; i++) {
        int count = 0;
        for (int j=0; j<arr.length; j++) {
                if (arr[i] == arr[j]) {
                        count ++;
                }
        }
        if (count > target) {
        boolean found = false;
        for (int j=0; j<i; j++) {
                if (arr[i] == arr[j]) {
                        found true;
                        break;
                }
        }
        if (!found) {
                sop (arr[i]);
        }
}
```

o/p = 1, 2.

Page No.
Date

* freq array :—

```
int arr[] = { 1, 1, 1, 2, 2, 2, 3}
int max = 0; int target = 2;
for (int i = 0; i < arr.length; i++){
        max = Math.max( arr [i], max);
  }
int freq [] = new int [max+1];
for (int i = 0; i < arr.length ; i++) {
        freq [ arr [i]] ++;
  }
for (int i = 0 ; i < freq.length; i++){
        if (freq [i] > target) {
                Sop (i);
        }
  }
```

O/p = 1 , 2 .

* HashMap :—

```
int target = 2;
HashMap <Integer, Integer> hm = new HashMap<>()
for (int i = 0; i < arr.length ; i++) {
        hm.put ( arr [i], hm.getOrDefault (
                arr[i], 0) + 1);
  }
for ( Map.Entry <Integer, Integer> entry :
                hm.entrySet ()) {
        if ( entry.getValue () > target) {
                Sop ( entry.getKey ());
  }
```

exactly k freq element :—

```
int arr [] = { 1, 1, 2, 2, 2, 3 }
int k = 2;
for (int i=0 ; i< arr.length ; i++) {
    int count = 0;
    for (int j=0 ; j< arr.length ; j++) {
        if (arr [i] == arr [j]) {
            Count ++;
        }
    }
    if (count == k) {
        boolean found = false;
        for (int j=0 ; j<i ; j++) {
            if (arr [i] == arr [j]) {
                found = true;
                break;
            }
        }
        if (!found) {
            sop (arr [i]);
        }
```

O/P = 1.

Page No.
Date

* freq array :-

```
int max = 0;   k = 2;
for (int i=0; i< arr.length ; i++) {
        max = Math.max ( arr[i], max);
  }
int freq [] = new int [max+1];
for (int i = 0 ; i< arr.length ; i++) {
        freq [ arr[i]] ++;
  }
for (int i = 0 ; i< freq.length ; i++) {
        if ( freq (i] == k) {
                sop (i);
        }
}
```

* HashMap :-

```
HashMap < Integer, Integer> hm = new HashMap
        <>();
for (int i = 0; i< arr.length ; i++) {
        hm.put (arr[i], hm.getOrDefault( arr(i), 0)+1)
  }
for (Map. Entry < Integer, Integer> entry : hm.entrySet()
        if ( entry. getValue ( ) == k) {
                Sop ( entry . getkey ( ));
        }
```

O/P = 1

3) element appearing only once - Unique element

```
int arr[] = { 1, 1, 2, 2, 2, 3 }
for (int i=0; i<arr.length; i++){
    int count = 0;
    for (int j=0; j<arr.length; j++){
        if (arr[i] == arr[j]){
            count ++;
        }
    }
    if (count == 1){
        boolean found = false;
        for(int j=0; j<i; j++){
            if (arr[i] == arr[j]){
                found = true;
                break;
            }
        }
        if (!found){
            Sop (arr[i]);
        }
    }
}
```

op = 3. unique element is only 3.

Page No.
Date

* freq array :-

```
int max = 0;
for (int i = 0; i<arr.length; i++){
        max = Math.max (arr[i], max);
}
int freq[] = new int [max+1];
for (int i=0; i< arr.length; i++){
        freq [arr[i]] ++;
}
for (int i=0; i< freq.length; i++){
        if (freq[i] == 1) {
                Sop (i);
        }
}
```

* HashMap :-

```
HashMap <Integer, Integer> hm = new HashMap <>();
for (int i = 0; i<arr.length; i++){
        hm.put (arr[i], hm.getOrDefault (arr[i],
                0)+1);
}
for (Map.Entry <Integer, Integer> entry :
                hm.entrySet ()) {
        if (entry.getValue () == 1) {
                Sop (entry.getKey());
        }
}
```

4) remove duplicate from array :-

```
int arr[] = {1, 2, 2, 3}
int newArr = new int [arr.length];
int index = 0
for (int i = 0; i < arr.length; i++) {
        int count = 0;
        for (int j = 0; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                        count++;
                }
        }
        if (count == 1) {
                newArr[index] = arr[i];
                index++;
        }
}
for (int i = 0; i < index; i++) {
        Sop (newArr[i]);
}
}
```

o/p = 1, 3.

array मध्ये return करायचाय. म्हणून new array घेतला.

freq :-

```
int max = 0;
for (int i = 0; i < arr.length ; i++){
        max = Math.max (arr[i], max);
}
int freq [] = new int [max+1];
for (int i = 0; i < arr.length; i++){
        freq [arr [i]] ++;
}
int unique [] = new int [arr.length];
int index = 0;
for (int i = 0 ; i < freq.length ; i++){
        if (freq [i] == 1) {
                unique [index] = i;
                index ++;
        }
}
for (int i = 0; i < index ; i++){
        sop (unique [i]);
}
```

Hash Map : —

```
int arr[] = {1,1,2,2,2,3}
ArrayList al = new ArrayList ();
HashMap <Integer, Integer> hm = new
HashMap <> ();
for (int i = 0; i<arr.length; i++){
    hm.put (arr[i], hm.getOrDefault(
        arr[i], 0) + 1);
}
for (Map.Entry <Integer, Integer> Entry
    hm.entrySet()){
        if (entry.getvalue () == 1){
            al.add (entry.getkey()
    }
}
Sop (al);
```

5) replace element with frequency.

[1, 2, 2, 3] [1, 2, 2, 1]

```
int arr[] = { 1, 2, 2, 3 }
for (int i = 0; i < arr.length; i++) {
        int count = 0;
        for (int j = 0; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                        count++;
                }
        }
        arr[i] = count;
}
for (int i = 0; i < arr.length; i++) {
        Sop (arr[i]);
}
}
```

O/P = [1, 2, 2, 1].

direct count print केला तर (1, 2, 1) 2 ची किंमत फक्त च घेतली जाते.
म्हणून तो count arr मध्ये store केला आणि array print केला.

freq array :-

```
int max = 0
for (int i = 0; i < arr.length; i++){
        max = Math.max( arr[i], max);
}
int freq[] = new int[max+1];
for (int i = 0; i < arr.length; i++){
        freq[arr[i]]++;
}
for (int i = 0; i < arr.length; i++){
        arr[i] = freq[arr[i]];
}
for (int i = 0; i < arr.length; i++){
        Sop(arr[i]);
}
}
```

Same last code सारख freq ही arr मध्ये store केली.

Page No.
Date

HashMap :-

```
HashMap <Integer, Integer> hm = new HashMap<>();
for (int i = 0; i < arr.length; i++) {
        hm.put (arr[i], hm.getOrDefault (arr[i], 0)
            +1);
    }
for (int i = 0; i < arr.length; i++) {
        arr[i] = hm.get (arr[i]);
    }
for (int i = 0; i < arr.length; i++) {
        Sop (arr[i]);
}
```

6) Find missing element between two array.

```
int arr[] = { 1, 2, 2, 2, 3, 7 }
int arr1[] = { 1, 2, 2, 3 }

for (int i=0; i< arr.length; i++) {
        boolean found = ~~true~~ false;
        for (int j=0; j< arr1.length; j++) {
                if (arr[i] == arr1[j]) {
                        ~~arr1[j]~~ found = true;
                        break;
                }
        }
}
if (!found) {
        Sop (arr[i]);
}
```

~~freq~~ :-

```
int max = 0;
for (int i=0; i< arr1.length; i++) {
        max = Math.max( arr[i], max)
}
int freq[] = new int [max+1];
for (int i=0; i< arr1.length; i++) {
        freq [arr1[i]] ++;
}
for (int i=0; i< arr.length; i++) {
        if (freq [arr[i]] == 0) {
                Sop (arr[i]);
        }
}
```

HashMap :-

```
HashMap <Integer,Integer> hm = new HashMap <>();
for ( int i = 0 ; i<arr.length ; i++){
        hm.put (arr [i] , hm.getOrDefaut (arr[i],0)
        +1);
}
for (int i=0 ; i < arr1.length ; i++){
        hm.put ( arr [i] , hm.getOrDefault (
        arr [i] , 0) -1);
}
for (Map.Entry <Integer,Integer> entry :
        hm.entrySet()) {
            if (entry.getValue ()>0) {
                    Sop ( entry.getkey())
}
```

Page No.
Date

check if String is panagram.

for Loop:-

```
String str = "abcdefghijklmnopqrstuvwxyz";
boolean found = true;
for (char ch = 'a'; ch <= 'z'; ch++){
        found = false;
    for (int i=0; i< str.length(); i++){
            if (str.length charAt(i) == c
                    found = true;
        }
    }
    if (!found){
        break;
    }
}
if (found){
        Sop (true);
}else{
        Sop (false);
}
```

Page No.
Date

* freq array :-

```
int freq [] = new int [26];
for (int i=0; i< Str.length (); i++){
        freq [ str.charAt (i)] ++;
}
boolean found = true;
for (int i=0 ; i< freq.length ; i++){
        if (freq[i] ==0){
                found = false ;
        }
}
if (found) {
        Sop (true);
}else {
        Sop (false);
}
```

* HashMap :-

```
HashMap < Character, Integer> hm = new HashMap <>();
for (int i=0 ; i< Str.length (); i++ ){
        if (str.charAt(i) >= 'a' && Str.charAt(i)
<= 'z') {
        hm.put ( str.charAt(i), hm.getOrDefault
                (Str.charAt(i), 0) +1);
}
if ( hm. size == 26) {
        Sop (true);
}else {
        Sop (false);
}
```

8) most frequent character in String = "apple";

```
int max = 0;
char maxchar = Str.charAt(0);
for (int i=0; i< Str.length(); i++) {
        int count = 0;
        for (int j=0; j< Str.length(); j++) {
                if (Str.charAt(i) == Str.charAt(j))
                        count ++;
        }
}
        if (count > max) {
                max = count;
                maxchar = Str.charAt(i);
        }
}
        Sop (maxchar);
```

o/p = p.

freq array :—

```
String Str = "apple";
int max = 0;
char maxchar = Str.charAt(0);
int freq [] = new int [26];
for (int i=0; i< Str.length (); i++) {
        freq [Str.charAt (i) - 'a'] ++;
}
for (int i=0; i< freq.length(); i++)
```

```
        if (freq [i] > max){
            max = freq[i];
            maxchar = (char) (i + 'a');
        }
    }
    Sop (maxchar);
}
o/p = p.
```

Hashmap :-

```
int max = 0;
char maxchar = str.charAt(0);
HashMap <Character, Integer> hm = new HashMap<>();
for (int i=0; i< str.length(); i++){
    hm.put ( str.charAt(i), hm.getOrDefault
            ( str.charAt (i), 0)+1);
}
for (Map.Entry <Character, Integer> entry :
            hm. entrySet ()){
    if (entry.getValue() > max) {
            max = entry.getValue();
            maxchar = entry.getkey();
    }
}
sop (maxchar);
}
o/p = p.
```

Page No.
Date

9) Second ~~list~~ least frequency element in array.

Ans:

```
int arr[] = {1,1,2,2,2,3}
int secondmin = Integer.MAX_VALUE;
int secondval = Integer.MAX_VALUE;
int min = Integer.MAX_VALUE;
int minval = Integer.MAX_VALUE;
for (int i=0; i< arr.length; i++){
        boolean found= false;
        for (int j=0; j< i; j++){
                if (arr[i] == arr[j]){
                        found = true;
                        break;
                }
        }
int count=0;
if (found){
        Continue;
}
for (int j=0; j<arr.length; j++){
        if (arr[i] ==arr[j]){
                Count ++;
        }
}
if (count <min){
        secondmin = min;
        Second val = minVal;
        min = count;
        minval = arr[i];
}
Sop (Secondval);
}
```

O/P = 1

Page No.
Date

freq :-

```
int freq [] = new int (max+1];
for (int i=0; i< arr.length; i++){
        freq [arr [i]] ++;
}
int secondmin = Integer. MAX_VALUE;
int secondval = Integer. MAX_VALUE;
int min = Integer. MAX_VALUE;
int minval = Integer. MAX_VALUE;
for (int i=0; i< freq.length; i++){
        if (freq[i]>0){
                if (freq[i] < min){
                        secondmin = min;
                        secondval = minval;
                        min = freq[i];
                        minval = i;
                }
        }
}
Sop (secondval);
}
```

Page No.
Date

* HashMap : —

```
int min = Integer.MAXVALUE;
int secondmin = Integer.MAX_VALUE;
HashMap <Integer, Integer> hm = new HashMap<>();
for (int i=0; i< arr.length; i++) {
        hm.put (arr[i], hm.getOrDefault (arr[i], 0)+1)
    }
for (Map.Entry <Integer, Integer> entry : hm.entrySet())
        int freq = ~~hm.~~entry.getValue();
        if (freq < min) {
                secondmin = min;
                min = freq;
        }
    }
for (Map.Entry <Integer, Integer> entry : hm.entrySet()
        if (entry.getValue() == secondmin) {
                Sop (entry.getkey());
        }
    }
}
```

o/p = 1.

Page No.
Date

10) find common elements.

Ans:

```
int arr1 [] = { 1,2,2,3 }
int arr2 [] = {2,2,4}
for (int i=0; i< arr1.length; i++){
    for (int j=0; j< arr2.length; j++){
        if ( arr1[i] == arr2[i]) {
            Sop ( arr[i]);
            break;
        }
    }.
```

O/p : 2, 2.

* freq :-

```
int freq [] = new int [100];
for (int i=0; i< arr1.length; i++){
    freq[ arr1[i]]++;
}
for (int i=0; i< arr2.length; i++){
    if (freq [arr2[i]] >0) {
        Sop (arr2[i]);
        freq [arr2[i]]--;
    }
}
```

O/p : 2, 2.

Page No.
Date

```
HashMap <Integer, Integer> hm = new HashMap<>();
 for ( int i=0; i< arr1.length; i++) {
      hm.put (arr1[i], hm.getOrDefault (arr1[i],0)+1)
 }
for ( int i=0; i< arr2.length; i++) {
     for (Map.Entry <Integer, Integer> hm: ent
                hm.entrySet ()) {
                    if (entry.getkey () == arr2[i] &&
                      entry.getvalue () > 0) {
                          Sop (entry.getkey());
     hm.put (entry.getkey (), entry.getvalue ()-1)
     }
}
```

11) check if two String are anagram.

Ans:

for loop :-

```
String str1 = "earth";
String str2 = "heart";
int count = 0;
for (int i=0; i< str1.length( ); i++){
    for (int j=0; j< str2.length( ); j++){
        if (str1.charAt(i) == str2.charAt(j))
        {
            Count ++;
            break;
        }
}
if (count == str1.length){
    Sop (true)
}else {
    Sop (false);
}
```

* O/p = true.

* freq array :-

```
int freq[]=new int[26];
for (int i=0; i< str1.length(); i++){
    freq [str1.charAt(i) - 'a']++;
}
for (int i=0; i< str2.length( ); i++){
    freq [str2.charAt (i) - 'a']--;
}
boolean found = true;
for (int i=0; i< freq.length; i++){
```

```
        if (freq[i] != 0){
            found = false;
        }
    }
    if (found){
        Sop ("anagram"); true;
    }else{
        Sop (false);
    }
}
```

HashMap: -

```
HashMap <Character, Integer> hm1 = new HashMap<>();
HashMap <Character, Integer> hm2 = new HashMap <>();
for (int i=0; i<s1.length(); i++){
        hm1.put(s1.charAt(i), hm.getOrDefault(
                s1.charAt(i), 0)+1);
    }
for (int i=0; i<s2.length(); i++){
        hm2.put(s2.charAt(i), hm.getOrDefault(
                s2.charAt(i), 0)+1);
    }
~~Sop~~ Sop(hm1.equals(hm2));
}
```

O/P : true.

Page No.
Date

Second highest frequency element :

~~for loop~~ :-

```
int max=0;
int second = 0;
for (int i = 0; i< arr.length; i++){
    int count = 0;
    for (int j=0; j< arr.length; j++){
        if (arr[i] == arr[j]){
            Count ++;
        }
    if (count > max){
        second = max;
        max = count;
    }
}

for (int j=0; j< arr.length ;j++){
    int count = 0;
    for (int j=0 ;j< arr.length; j++){
        if (arr[i] == arr[j]){
            Count ++;
        }
    if (count == second){
        Sop (arr[i]);
    }
}
```

O/P = 2 or 4.

* freq array :-

```
int max = 0
for (int i=0; i< arr.length ; i++){
        max = Math.max ( arr[i], max);
}
int freq[] = new int [max+1];
for (int i=0; i< arr.length ; i++){
        freq [arr[i]] ++;
}
int maxele = 0;
int second = 0;
for (int i=0; i< freq.length; i++){
        if (freq [i] > max){
                second = max;
                max = freq[i];
        }
}
for (int i=0; i< freq.length; i++)
        if (freq [i] == second) {
                SOP (i);
        }
}
```

o/p = 2, 4

Page No.
Date

HashMap :-

```
HashMap <Integer, Integer> hm = new HashMap<>();
for (int i = 0; i < arr.length; i++){
        hm.put (arr[i], hm.getOrDefault (arr[i], 0)+1);
  }
int max = 0;
int second = 0;
for (Map.Entry <Integer, Integer> entry : hm.entrySet()){
        int freq = entry.getValue();
        if (freq > max){
                Second = max;
                max = freq;
        }
  }
for (Map.Entry <Integer, Integer> entry : hm.entrySet()
        if (entry.getValue() == second) {
                Sop (entry.getKey());
        }
}
```